In [122…
```
# import numerical libraries
import numpy as np
import pandas as pd
import warnings

# import graphical plot libraries
import seaborn as sn
import matplotlib.pyplot as plt
%matplotlib inline

# Import Linear regression machine learning libraries
from sklearn import preprocessing
from sklearn.preprocessing import PolynomialFeatures
from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import r2_score
```

In [124…
```
warnings.filterwarnings('ignore')
# read / load data from csv.
data = pd.read_csv(f"C:\sample\datafiles\car-mpg.csv")
data.head()
```

Out[124…

	mpg	cyl	disp	hp	wt	acc	yr	origin	car_type	car_name
0	18.0	8	307.0	130	3504	12.0	70	1	0	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.5	70	1	0	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	70	1	0	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	70	1	0	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	70	1	0	ford torino

In [126…
```
#Drop car name
#replace origin 1,2,3...dont forget get dummies
#Replace ? with nan
#Replace all nan with median


#data = data.drop(['car_name'], axis = 1)
#data['origin'] = data['origin'].replace({1:'america', 2:'europe', 3:'asia'})
#data = pd.get_dummies(data,columns = ['origin'])
#data = data.replace('?',np.nan)
#data = data.apply(lambda x: x.fillna(x.median()), axis = 0)

data = data.drop(['car_name'], axis = 1)
data['origin'] = data['origin'].replace({1: 'america', 2: 'europe', 3: 'asia'})
data = pd.get_dummies(data,columns = ['origin'])
data = data.replace('?', np.nan)
# Identify columns that should be numeric but might contain NaNs or be object type.
# It's good practice to list your numeric columns or infer them.
# For example, if 'horsepower' and 'acceleration' were originally numbers but had
numeric_cols = ['cyl', 'disp', 'hp', 'wt', 'acc', 'yr']
for col in numeric_cols:
    # pd.to_numeric will convert to a number, coercing errors to NaN
    # errors='coerce' is crucial here: it turns any value that *cannot* be converted
    # into a number into NaN. This handles cases where '?' might not have been replaced,
    # or other non-numeric strings exist.
    data[col] = pd.to_numeric(data[col], errors='coerce')
data = data.apply(lambda x: x.fillna(x.median()), axis = 0)
```

In [139…
```
data.head()
```

Out[139…

	mpg	cyl	disp	hp	wt	acc	yr	car_type	origin_america	origin_asia	origin_europe
0	18.0	8	307.0	130.0	3504	12.0	70	0	True	False	False
1	15.0	8	350.0	165.0	3693	11.5	70	0	True	False	False
2	18.0	8	318.0	150.0	3436	11.0	70	0	True	False	False
3	16.0	8	304.0	150.0	3433	12.0	70	0	True	False	False
4	17.0	8	302.0	140.0	3449	10.5	70	0	True	False	False

Aim here is the predict the mileage column given

In [142…
```
X = data.drop(['mpg'], axis=1) # independent variable
Y = data[['mpg']] #dependtent variable
print(X)
print(Y)
```

```
     cyl   disp     hp    wt   acc  yr  car_type  origin_america  origin_asia  \
0      8  307.0  130.0  3504  12.0  70         0            True        False
1      8  350.0  165.0  3693  11.5  70         0            True        False
2      8  318.0  150.0  3436  11.0  70         0            True        False
3      8  304.0  150.0  3433  12.0  70         0            True        False
4      8  302.0  140.0  3449  10.5  70         0            True        False
..   ...    ...    ...   ...   ...  ..       ...             ...          ...
393    4  140.0   86.0  2790  15.6  82         1            True        False
394    4   97.0   52.0  2130  24.6  82         1           False        False
395    4  135.0   84.0  2295  11.6  82         1            True        False
396    4  120.0   79.0  2625  18.6  82         1            True        False
397    4  119.0   82.0  2720  19.4  82         1            True        False

     origin_europe
0            False
1            False
2            False
3            False
4            False
..             ...
393          False
394           True
395          False
396          False
397          False

[398 rows x 10 columns]
      mpg
0    18.0
1    15.0
2    18.0
3    16.0
4    17.0
..    ...
393  27.0
394  44.0
395  32.0
396  28.0
397  31.0

[398 rows x 1 columns]
```

In [144…
```
#Scaling the dataset

X_s = preprocessing.scale(X)
X_s = pd.DataFrame(X_s, columns = X.columns) #converting scaled data into dataframe

Y_s = preprocessing.scale(Y)
Y_s = pd.DataFrame(Y_s, columns = Y.columns) # ideally train and test data
```

In [146…
```
X_train, X_test, Y_train, Y_test = train_test_split(X_s, Y_s, test_size=0.30,random_state=1)
X_train.shape
```

Out[146… (278, 10)

In [148…
```
#Fit simple linear model and find coefficients
regression_model = LinearRegression()
regression_model.fit(X_train, Y_train)

for idx, col_name in enumerate(X_train.columns):
    print('The coefficient for {} is {}'.format(col_name, regression_model.coef_[0][idx]))

intercept = regression_model.intercept_[0]
print('The intercept is {}'.format(intercept))
```

```
The coefficient for cyl is 0.321022385691611
The coefficient for disp is 0.32483430918483897
The coefficient for hp is -0.22916950059437569
The coefficient for wt is -0.7112101905072298
The coefficient for acc is 0.014713682764191237
The coefficient for yr is 0.3755811949510748
The coefficient for car_type is 0.3814769484233099
The coefficient for origin_america is -0.07472247547584178
The coefficient for origin_asia is 0.044515252035677896
The coefficient for origin_europe is 0.04834854953945386
The intercept is 0.019284116103639764
```

In [150…
```
ridge_model = Ridge(alpha =0.3)
ridge_model.fit(X_train, Y_train)

print('Ridge coef.{}'.format(ridge_model.coef_))
```

```
Ridge coef.[[ 0.31649043  0.31320707 -0.22876025 -0.70109447  0.01295851  0.37447352
   0.37725608 -0.07423624  0.04441039  0.04784031]]
```

In [155…
```
lasso_model = Lasso(alpha =0.1)
lasso_model.fit(X_train, Y_train)

print('Lasso coef.{}'.format(lasso_model.coef_))
```

```
Lasso coef.[-0.         -0.         -0.01690287 -0.51890013  0.          0.28138241
  0.1278489  -0.01642647  0.          0.        ]
```

**Score comparision

In [160…
```
#Model score - r^2 or coeff of determinant
#r^2 = 1-(RSS/TSS) = Regression error/TSS


#Simple Linear Model
print(regression_model.score(X_train, Y_train))
print(regression_model.score(X_test, Y_test))
```

```
print('*************************')
#Ridge
print(ridge_model.score(X_train, Y_train))
print(ridge_model.score(X_test, Y_test))

print('*************************')
#Lasso
print(lasso_model.score(X_train, Y_train))
print(lasso_model.score(X_test, Y_test))
```

```
0.8343770256960538
0.8513421387780066
*************************
0.8343617931312616
0.8518882171608506
*************************
0.7938010766228453
0.8375229615977083
```

In [170…
```
#Model parameter tuning

data_train_test = pd.concat([X_train, Y_train], axis =1)
data_train_test.head()
```

Out[170…

	cyl	disp	hp	wt	acc	yr	car_type	origin_america	origin_asia	origin_europe	mpg
350	-0.856321	-0.849116	-1.081977	-0.893172	-0.242570	1.351199	0.941412	0.773559	-0.497643	-0.461968	1.432898
59	-0.856321	-0.925936	-1.317736	-0.847061	2.879909	-1.085858	0.941412	-1.292726	-0.497643	2.164651	-0.065919
120	-0.856321	-0.695475	0.201600	-0.121101	-0.024722	-0.815074	0.941412	-1.292726	-0.497643	2.164651	-0.578335
12	1.498191	1.983643	1.197027	0.934732	-2.203196	-1.627426	-1.062235	0.773559	-0.497643	-0.461968	-1.090751
349	-0.856321	-0.983552	-0.951000	-1.165111	0.156817	1.351199	0.941412	-1.292726	2.009471	-0.461968	1.356035

In [174…
```
import statsmodels.formula.api as smf
ols1 = smf.ols(formula = 'mpg ~ cyl+disp+hp+wt+acc+yr+car_type+origin_america+origin_europe+origin_asia', data = data_train_test).fit()
ols1.params
```

Out[174…
```
Intercept         0.019284
cyl               0.321022
disp              0.324834
hp               -0.229170
wt               -0.711210
acc               0.014714
yr                0.375581
car_type          0.381477
origin_america   -0.074722
origin_europe     0.048349
origin_asia       0.044515
dtype: float64
```

In [179…
```
print(ols1.summary())
```

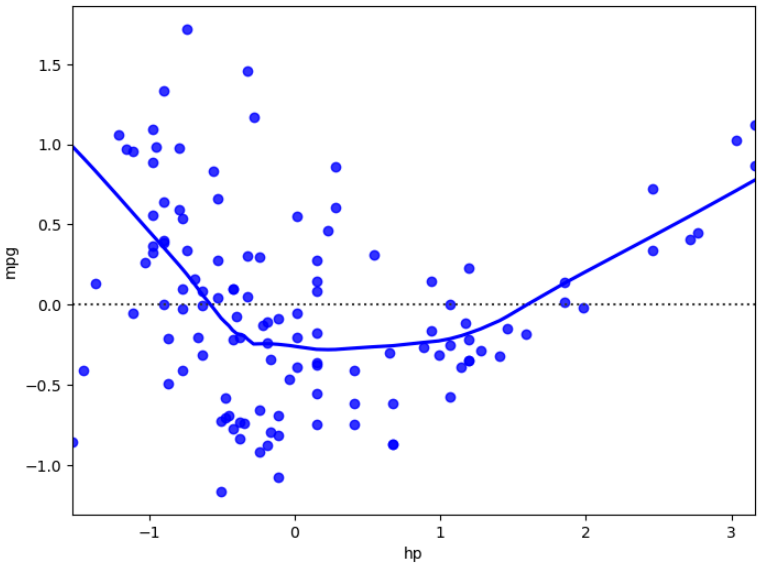
```
                            OLS Regression Results
==============================================================================
Dep. Variable:                    mpg   R-squared:                       0.834
Model:                            OLS   Adj. R-squared:                  0.829
Method:                 Least Squares   F-statistic:                     150.0
Date:                Mon, 14 Jul 2025   Prob (F-statistic):           3.12e-99
Time:                        21:45:10   Log-Likelihood:                -146.89
No. Observations:                 278   AIC:                             313.8
Df Residuals:                     268   BIC:                             350.1
Df Model:                           9
Covariance Type:            nonrobust
==================================================================================
                     coef    std err          t      P>|t|      [0.025      0.975]
----------------------------------------------------------------------------------
Intercept          0.0193      0.025      0.765      0.445      -0.030       0.069
cyl                0.3210      0.112      2.856      0.005       0.100       0.542
disp               0.3248      0.128      2.544      0.012       0.073       0.576
hp                -0.2292      0.079     -2.915      0.004      -0.384      -0.074
wt                -0.7112      0.088     -8.118      0.000      -0.884      -0.539
acc                0.0147      0.039      0.373      0.709      -0.063       0.092
yr                 0.3756      0.029     13.088      0.000       0.319       0.432
car_type           0.3815      0.067      5.728      0.000       0.250       0.513
origin_america    -0.0747      0.020     -3.723      0.000      -0.114      -0.035
origin_europe      0.0483      0.021      2.270      0.024       0.006       0.090
origin_asia        0.0445      0.020      2.175      0.031       0.004       0.085
==============================================================================
Omnibus:                       22.678   Durbin-Watson:                   2.105
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               36.139
Skew:                           0.513   Prob(JB):                     1.42e-08
Kurtosis:                       4.438   Cond. No.                     1.59e+16
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The smallest eigenvalue is 6.14e-30. This might indicate that there are
strong multicollinearity problems or that the design matrix is singular.
```

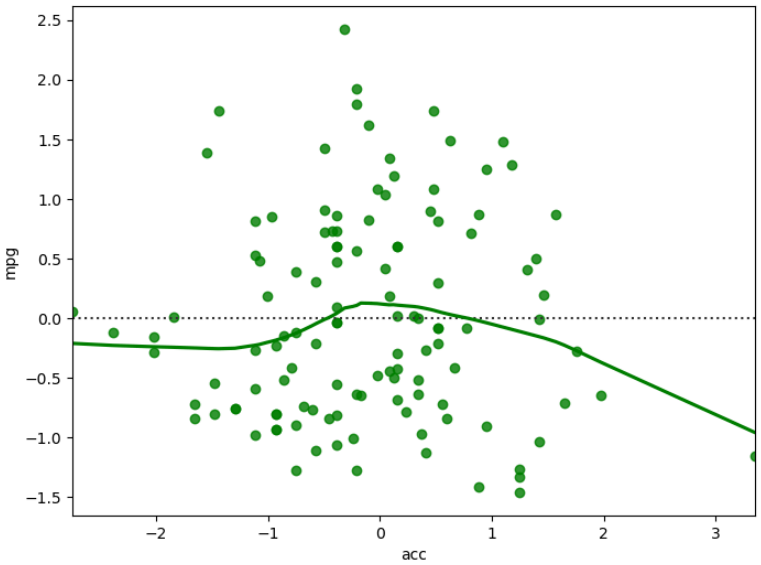
In [184…
```
#Lets check Sum of Squared Errors (SSE) by predicting value of y for test cases and subtracting from the actual y for the test cases
mse  = np.mean((regression_model.predict(X_test)-Y_test)**2)

# root of mean_sq_error is standard deviation i.e. avg variance between predicted and actual
import math
rmse = math.sqrt(mse)
print('Root Mean Squared Error: {}'.format(rmse))
```

```
Root Mean Squared Error: 0.37766934254087847
```

In [197…
```
# Is OLS a good model ? Lets check the residuals for some of these predictor.
```

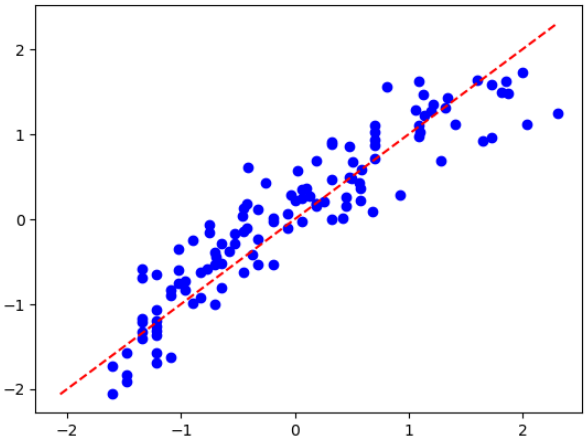
```
fig = plt.figure(figsize=(8,6))
sn.residplot(x= X_test['hp'], y= Y_test['mpg'], color='blue', lowess=True )


fig = plt.figure(figsize=(8,6))
sn.residplot(x= X_test['acc'], y= Y_test['mpg'], color='green', lowess=True )
plt.show()
```





In [228…

```
y_pred = regression_model.predict(X_test)

min_value = min(Y_test['mpg'].min(), y_pred.min())
max_value = max(Y_test['mpg'].max(), y_pred.max())

plot_range = np.linspace(min_value,max_value, 100)

plt.plot(plot_range, plot_range, color='red', linestyle='--', label='Perfect Prediction (y=x)')
plt.scatter(Y_test['mpg'], y_pred, color='blue')
plt.show()
```



In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: